

个人报告：昇腾端AI视觉子系统开发与调优

姓名：林棉峰 选题：题目4 · 昇腾310B目标跟踪检测 主要负责：昇腾端检测模型调试与优化、系统联调 提交日期：2026-07-13

一、个人工作概述

在本项目中，我主要负责昇腾310B端AI视觉子系统的开发与优化，具体包括：

- MobileNet-SSD检测链路完整源码阅读与理解
- NPU推理参数调优（置信度阈值、NMS阈值等）
- 摄像头采集优化（V4L2驱动、MJPEG格式、缓冲区控制）
- 串口输出模块（GimbalSerial + TargetLock）的设计与实现
- 模型输出兼容性处理（4种格式）
- 检测→跟踪→串口全链路联调

我的工作核心是确保昇腾端的"眼睛"看得准、看得快、说得出——检测精度够、推理速度快、坐标输出稳。

二、检测链路深度理解

2.1 源码通读与分析

项目开始阶段，我花了约一天半时间逐层通读了case2的全部检测相关源码。这不是"跑起来就行"的浅层理解，而是从上到下理清了每一层的职责和数据流转。

代码阅读路径：

```
入口层：scripts/detection_app.py → scripts/tracking_app.py
↓ 参数解析、后端选择、主循环编排
检测核心层：ssdlite/backend_base.py
↓ 预处理(preprocess_frame) → 推理(_run_model) → 解码(decode_detections)
NPU后端层：ssdlite/npu_backend.py
↓ ACL初始化 → 模型加载 → Device内存分配 → H2D→execute→D2H
```

解码层: ssdlite/decoder.py
↓ DefaultBoxes生成 → scale_back_batch → decode_single → NMS

关键理解点：

模块	核心理解
<code>preprocess_frame</code>	BGR→RGB→resize 320×320→归一化 [0,1]→ImageNet标准化。必须和训练时一致！
<code>NpuBackend._run_model</code>	ACL三层调用： <code>memcpy</code> H2D→ <code>mdl.execute</code> → <code>memcpy</code> D2H。输入必须是 <code>np.float32</code> 连续数组
<code>SSDDecoder.scale_back_batch</code>	网络输出的是 相对于先验框的偏移量 ，需要反变换： <code>cx = offset_cx × scale_xy × dbox_w + dbox_cx</code>
<code>decode_single</code>	先为每个prior选最佳前景类别→分数筛选→按类NMS。不是"每类扫全部prior"的低效方式
<code>_nms</code>	按分数排序→取最高分→计算IOU→删除高重叠框。关键是 <code>_calc_iou_with_box</code> 专门优化"多框对单框"

2.2 四种模型输出格式的兼容处理

这是我在阅读`decode_detections`函数时发现的一个重要工程细节——不同模型（MobileNet不同版本、ONNX不同导出方式）的输出格式**并不统一**。

`backend_base.py`中的`decode_detections`函数兼容了四种格式：

格式	特点	识别方式
SSD先验框格式	boxes shape (4, 3234) 或 (3234, 4)，需SSDDecoder解码	根据prior数量匹配
TF Detection 格式	字典: <code>detection_boxes</code> , <code>detection_scores</code> , <code>detection_classes</code> , <code>num_detections</code>	按key名匹配
通用格式 (6列)	<code>[x1,y1,x2,y2,score,class_id]</code>	<code>tensor.shape[-1]==6</code>

格式	特点	识别方式
紧凑格式 (7列)	<code>[img_id,class_id,score,x1,y1,x2,y2]</code>	<code>tensor.shape[-1]==7</code> 且第0列 ≈ 0

这个兼容层的意义在于：切换不同MobileNet版本或ResNet骨干时，**不需要改解码代码**，大大降低了调试成本。

三、检测参数调优

3.1 置信度阈值（score_threshold）调优

初始值：0.5（教程默认） 最终值：0.35

调优过程：

我通过反复实验发现了默认阈值0.5的问题——在室内光线条件下，稍远一点的行人置信度在0.4~0.5之间，这导致了**间歇性漏检**。跟踪器依赖检测器输出，漏检会直接导致轨迹断开。

我把阈值降到0.35后的效果：

- 检测框数量从平均2~3个增加到4~6个
- 远处目标不再漏检
- 虽然有少量低分假阳性，但跟踪器的`min_hits`和`max_age`机制会自动过滤短期误检
- 对跟踪稳定性有明显改善

核心认识：在AI推理→跟踪的链路中，**宁可多检不可漏检**。跟踪器自带轨迹管理，短暂假阳性会被自动淘汰，但漏检导致的轨迹断裂很难恢复。

3.2 NMS阈值调优

NMS阈值保持默认0.45，经过实验验证在此场景下表现合适：

- 室内单人或少数目标场景，IOU重叠不严重
- 0.45能有效去重，又不会误删相邻的不同目标
- 降低到0.3会出现相邻目标被误合并；升高到0.6会出现重复框

四、摄像头实时采集优化

4.1 问题发现

在联调初期，我发现即使是NPU推理只要~15ms，整体帧率也只有~20FPS。通过耗时统计系统（`infer_with_profile`的5阶段计时），我定位到瓶颈在**摄像头读取阶段**：

阶段	耗时	占比
Read	~30ms	60% ← 瓶颈！
Pre	~2ms	4%
Infer	~15ms	30%
Decode	~2ms	4%
Draw	~1ms	2%

4.2 优化方案与效果

我做了三件事：

① V4L2后端优先

OpenCV默认的摄像头后端在不同平台上行为不一致。我在`utils/opencv_runtime.py`中实现了V4L2优先策略：

```
# 优先使用 V4L2 后端打开摄像头（Linux下延迟更低）
cap = cv2.VideoCapture(index, cv2.CAP_V4L2)
```

② MJPEG格式

大多数USB摄像头在YUYV格式下只能达到640×480@30fps，但MJPEG模式下可达更高帧率。启用MJPEG后，摄像头内部完成JPEG压缩，传输数据量更小。

③ buffer=1减少积压

```
cap.set(cv2.CAP_PROP_BUFFERSIZE, 1) # 只保留最新帧，避免处理积压
```

默认缓冲区通常为3~4帧，当推理耗时大于帧间隔时，缓冲区会积压旧帧，导致画面延迟越来越大。buffer=1确保每次都拿到最新帧。

优化效果：

指标	优化前	优化后	提升
Read耗时	~30ms	~18ms	40% ↓
整体FPS	~20	~26	30% ↑

4.3 OpenCV运行时环境修复

部分OrangePi Alpro环境上cv2.imshow会因找不到Qt字体而崩溃。我在opencv_runtime.py中添加了自动修复逻辑：

- 设置QT_QPA_FONTDIR环境变量指向系统字体目录
- 自动创建字体目录符号链接

五、串口输出模块设计与实现

5.1 设计思路

串口输出模块是昇腾端和STM32端的桥梁。设计时我考虑了以下几个工程原则：

1. **归一化坐标**：不管摄像头实际分辨率多少，发给STM32的坐标始终在`[0,320]×[0,240]`范围内。这样STM32端的`CENTER_X=160`，`CENTER_Y=120`和`Kp`参数不需要随分辨率改变。
2. **限频发送**：STM32控制周期10ms，串口发送太快没有意义。我们限频20Hz（50ms间隔），与看门狗200ms超时匹配。
3. **无目标不发**：当画面中没有检测到任何目标时，不做任何发送。STM32端的200ms看门狗会自动触发急刹——这是通信丢失和语义丢失的统一处理。

5.2 GimbalSerial类实现

核心设计：

每帧调用 `send_target(boxes, frame_w, frame_h)`:

- ① 限频检查: 距上次发送不足50ms则跳过
- ② 选面积最大目标 (离摄像头最近的人通常框最大)
- ③ 计算中心坐标 (`cx`, `cy`)
- ④ 归一化 → 虚拟画面 320×240
- ⑤ EMA低通滤波 (`smooth=0.4`): 减少逐帧坐标抖动
- ⑥ 钳位到合法范围
- ⑦ 串口发送 `"X{nx}Y{ny}\n"`

EMA低通滤波是平滑的关键: $sx = 0.4 \times new_x + 0.6 \times old_sx$ 。检测框逐帧会有微小抖动, 如果每个像素偏差都发给STM32, 电机会在中心附近频繁微调, 画面会抖。经过EMA平滑后, 高频抖动被滤除, 云台运动更加流畅。

目标丢失时清空平滑状态 (`self._sx = None`): 新目标出现时直接对准, 不让旧目标的平滑值"拖影"过来。

5.3 TargetLock目标定格功能

在多目标场景中直接用"面积最大"选目标有一个问题: 如果两个人交叉走过, 当前最大目标可能在ID之间跳变, 导致云台突然转向另一个人。

TargetLock解决思路:

锁定流程:

- ① 有锁定目标 → 在所有存活轨迹里找它
 - 刚被检测到(`time_since_update==0`) → 用实测框
 - 短暂丢失(`≤max_coast=8`帧) → 保持最后位置, 不追预测值
 - 丢失超时 → 暂停发送 (云台停住等待), 但保留锁
 - 被跟踪器彻底删除(`>max_age`) → 清锁
- ② 无锁定目标 → 从当前确认轨迹中选最大的 → 锁定它

"保持最后位置而不是追卡尔曼预测" 这个设计很关键。目标短暂被遮挡时, 卡尔曼滤波器会按匀速模型外推, 但外推值可能跑出画面、跑到其他区域。保持最后位置等目标回来, 比追预测值更安全。

六、全链路联调经验

6.1 耗时统计系统

我在联调过程中充分利用了`backend_base.py`中`infer_with_profile`方法的5阶段计时。这个系统让我们能精准定位瓶颈：

Read	: 18.2ms	← 摄像头读取+解码
Pre	: 2.1ms	← 图像预处理
Infer	: 15.3ms	← NPU推理
Decode	: 2.8ms	← SSD解码+NMS
Draw	: 1.5ms	← 可视化绘制

Total	: 39.9ms	→ 约25 FPS

这种分阶段计时对于边缘设备的性能分析非常重要——每个阶段的瓶颈成因不同，需要分别优化。

6.2 遇到的典型问题

问题1：串口数据STM32收到的坐标偶尔错位

定位：昇腾端发送`X160Y120\n`，STM32偶尔解析出奇怪的值。

原因：串口物理连接在调试时容易被碰松，或者TX/RX线序不对。

解决：在GimbalSerial中添加`print`日志确认每帧发送的实际字符串；STM32端通过LED闪烁确认收到了数据。

问题2：模型切换时解码报错

定位：从mobilenetv1切换到mobilenetv4后，`decode_detections`报"无法识别模型输出格式"。

原因：不同版本的MobileNet在ONNX导出时输出tensor的shape和命名不同。

解决：深入理解`decode_detections`的四种格式兼容分支（参见第二节），确认mobilenetv4走的是SSD先验框路径。

七、个人收获与反思

7.1 技术收获

1. **边缘AI部署的完整链路**：从ONNX模型→ATC转换→OM模型→ACL推理→解码→跟踪，我完整理解了NPU部署的每一个环节。不是"调包"，而是逐层理解了ACL的Device内存管理、H2D/D2H传输和模型执行。
2. **SSD检测的底层原理**：通过阅读`decoder.py`，我深刻理解了为什么SSD叫"Single Shot"——它在一层特征图上同时做分类和回归，先验框只是参考模板，网络学习的是偏移量。
3. **实时系统的性能分析方法**：5阶段分步计时是很好的实践。只有分阶段测量，才能知道"慢在哪里"——这个项目里摄像头读取一度占了60%的时间，不拆分就永远在错的方向上优化。
4. **跨处理器通信的鲁棒性设计**：归一化坐标、限频发送、无目标不发+看门狗急刹，这些设计让两个独立运行的系统在通信层有了优雅的耦合。

7.2 可以改进的地方

1. **检测模型可以进一步轻量化**：MobileNetV4虽然在v3基础上优化了硬件感知设计，但如果只跟踪person类，可以考虑用更小的专用模型（如NanoDet），推理速度还能提升。
2. **串口协议可以加校验**：当前纯ASCII+\n帧同步够用，但在长距离或干扰环境下，加一个简单的异或校验会更可靠。
3. **EMA平滑参数可以按场景自适应**：当前`smooth=0.4`是固定的，目标快速移动时应该减小平滑、快速响应；目标静止时可以增大平滑、减少抖动。

八、总结

我在本项目中的核心贡献是：**让昇腾310B端从"能检测"变成"检测得又好又快又稳定，还能把结果准确地发出去"**。从深入理解源码到参数精调，从摄像头优化到串口模块开发，每一步都面对实际的工程问题并逐一解决。

最让我有成就感的是解决了摄像头卡顿问题——从20FPS提升到26FPS，这30%的提升不是改模型结构得来的，而是通过理解V4L2驱动、MJPEG格式和缓冲区机制，在系统层面做的优化。这让我意识到：**边缘AI的性能瓶颈往往不在AI本身，而在数据和通信路径上。**
